
Ottimizzazione su Reti

Capitolo 7

Il problema dei cammini minimi

Corso di Laurea Magistrale in Ingegneria Gestionale

Sapienza Università di Roma

Giampaolo Liuzzi

Dipartimento di Ingegneria Informatica, Automatica e Gestionale

Indice

1	Il problema del cammino minimo	1
2	Formulazione come flusso a costo minimo	1
3	Da una sorgente a tutti i nodi	2
4	Rilassamento e predecessori	2
5	Cammini minimi nei DAG	2
6	Algoritmo di Dijkstra	3
7	Algoritmo di Bellman-Ford	3
8	Cicli negativi e illimitatezza	4
9	Albero dei cammini minimi	4
10	Esempio comparativo	4
11	Python e NetworkX	5
12	Registro delle notazioni	5
13	Errori frequenti	5
14	Esercizi	6
15	Sintesi del capitolo	6

Obiettivi di apprendimento

Al termine del capitolo lo studente dovrebbe essere in grado di:

- formulare il cammino minimo come flusso a costo minimo;
- interpretare le variabili duali come etichette di distanza;
- applicare correttamente l'operazione di rilassamento;
- calcolare cammini minimi su un DAG;
- scegliere tra Dijkstra e Bellman-Ford in base ai costi degli archi;
- riconoscere e interpretare un ciclo di costo negativo;
- ricostruire i cammini mediante i predecessori e usare NetworkX.

1 Il problema del cammino minimo

Sia $G = (\mathcal{V}, \mathcal{E})$ un grafo orientato e sia $c_{ij} \in \mathbb{R}$ il costo, la lunghezza o il tempo associato all'arco (i, j) . Dati un'origine s e una destinazione t , il costo di un cammino orientato P è

$$C(P) = \sum_{(i,j) \in P} c_{ij}.$$

Il problema consiste nel determinare

$$\min\{C(P) : P \text{ è un cammino orientato da } s \text{ a } t\}.$$

In un grafo non orientato ogni spigolo $\{i, j\}$ può essere sostituito da due archi (i, j) e (j, i) , eventualmente con costi diversi.

Attenzione

Se un ciclo di costo negativo è raggiungibile da s e da esso è raggiungibile t , il problema della passeggiata minima s - t è illimitato inferiormente: ripetendo il ciclo il costo tende a $-\infty$. In assenza di tali cicli, esiste un cammino minimo semplice.

2 Formulazione come flusso a costo minimo

Inviare un'unità di flusso da s a t produce la formulazione

$$\min c^\top x \quad \text{s.t.} \quad Ax = b, \quad x \geq 0, \quad (1)$$

dove, coerentemente con NetworkX,

$$b_s = -1, \quad b_t = 1, \quad b_i = 0 \quad i \notin \{s, t\}.$$

La totale unimodularità della matrice di incidenza garantisce, per dati interi, l'esistenza di una soluzione di base intera. In assenza di cicli negativi, una soluzione ottima può essere scelta come vettore di incidenza di un cammino s - t .

Il duale di (1) è

$$\max d_t - d_s \quad \text{s.t.} \quad d_j - d_i \leq c_{ij} \quad \forall (i, j) \in \mathcal{E}. \quad (2)$$

Poiché i potenziali sono definiti a meno di una costante, poniamo $d_s = 0$. I vincoli diventano

$$d_j \leq d_i + c_{ij}.$$

Il massimo valore ammissibile di d_t è esattamente la distanza minima da s a t .

Debole dualità lungo un cammino

Sommando i vincoli duali sugli archi di un cammino P da s a t si ottiene

$$d_t - d_s \leq C(P).$$

Ogni vettore di etichette ammissibile fornisce dunque un limite inferiore al costo di ogni cammino. Quando l'uguaglianza vale sugli archi del cammino scelto, primale e duale certificano l'ottimalità.

3 Da una sorgente a tutti i nodi

Per calcolare simultaneamente le distanze da s a tutti i nodi, si può inviare un'unità a ciascun nodo:

$$b_s = -(n-1), \quad b_i = 1 \quad i \neq s.$$

Il duale diventa

$$\max \sum_{i \neq s} d_i \quad \text{s.t.} \quad d_j \leq d_i + c_{ij}, \quad d_s = 0.$$

Le etichette ottime d_i^* sono le distanze minime da s . La formulazione spiega perché gli stessi potenziali compaiono nel simpleso su reti e negli algoritmi dei cammini minimi.

Proposizione 3.1 (Sottostruttura ottima). *Ogni sottocammino di un cammino minimo è a sua volta un cammino minimo tra i suoi estremi.*

Se un cammino minimo verso j termina con l'arco (i, j) , allora

$$d_j^* = d_i^* + c_{ij}.$$

Pertanto, per ogni nodo raggiungibile $j \neq s$, vale l'equazione di Bellman

$$d_j^* = \min_{(i,j) \in \mathcal{E}} \{d_i^* + c_{ij}\}. \quad (3)$$

4 Rilassamento e predecessori

Gli algoritmi mantengono etichette d_j che rappresentano i migliori costi finora noti. Rilassare l'arco (i, j) significa eseguire

$$\text{se } d_j > d_i + c_{ij}, \quad d_j \leftarrow d_i + c_{ij}, \quad \pi_j \leftarrow i.$$

Il predecessore π_j consente di ricostruire il cammino risalendo da j fino a s . In caso di parità, predecessori diversi possono descrivere cammini minimi differenti.

Un singolo rilassamento

Se $d_i = 7$, $c_{ij} = 3$ e $d_j = 13$, passando per i si ottiene $10 < 13$: la nuova etichetta è $d_j = 10$ e il predecessore diventa $\pi_j = i$. Se invece $d_j = 9$, l'arco non produce alcun miglioramento.

5 Cammini minimi nei DAG

Sia G un grafo orientato aciclico e sia $v_1, \dots, v_n$ un ordinamento topologico con $v_1 = s$. Si inizializza $d_s = 0$ e $d_v = +\infty$ per $v \neq s$, quindi si rilassano gli archi uscenti dai nodi nell'ordine topologico.

Quando si elabora v_k , tutti i suoi possibili predecessori sono già stati elaborati; l'etichetta di v_k è quindi definitiva. La complessità, inclusa la numerazione topologica, è

$$O(|\mathcal{V}| + |\mathcal{E}|).$$

L'algoritmo è corretto anche con costi negativi, perché un DAG non contiene cicli.

Struttura	Costi ammessi	Complessità
DAG	anche negativi	$O(n + m)$
grafo generale, costi non negativi	$c_{ij} \geq 0$	Dijkstra
grafo generale	anche negativi	Bellman-Ford

6 Algoritmo di Dijkstra

Dijkstra richiede costi non negativi. Mantiene un insieme S di nodi con etichetta permanente e un insieme $T = \mathcal{V} \setminus S$ di nodi con etichetta provvisoria.

1. Porre $d_s = 0$, $d_v = +\infty$ per $v \neq s$, $S = \emptyset$.
2. Scegliere in T un nodo j di etichetta minima.
3. Rendere permanente d_j e porre $S \leftarrow S \cup \{j\}$.
4. Rilassare tutti gli archi (j, i) con $i \in T$.
5. Ripetere finché T è vuoto o la minima etichetta provvisoria è infinita.

Teorema 6.1 (Correttezza di Dijkstra). *Se tutti i costi sono non negativi, quando l'etichetta di un nodo diventa permanente essa coincide con la sua distanza minima da s .*

Idea della dimostrazione. Se esistesse un cammino più corto verso il nodo scelto j , tale cammino dovrebbe attraversare per la prima volta il confine tra S e T . Il primo nodo $q \in T$ sul cammino avrebbe un'etichetta non maggiore del costo del prefisso e, per non negatività, non maggiore della distanza alternativa verso j . Ciò contraddice la scelta di j come etichetta minima.

Con una coda di priorità binaria, la complessità è $O((n+m) \log n)$; con una semplice scansione delle etichette è $O(n^2 + m)$.

Attenzione

Dijkstra può produrre risultati errati in presenza di un arco di costo negativo, anche se non esistono cicli negativi. Il problema non è il ciclo: viene meno la garanzia che un'etichetta resa permanente non possa più diminuire.

7 Algoritmo di Bellman-Ford

Bellman-Ford ammette costi negativi e rileva cicli negativi raggiungibili dalla sorgente. Inizializza $d_s = 0$, $d_v = +\infty$ per $v \neq s$ e ripete il rilassamento di tutti gli archi per $n - 1$ passate.

Proposizione 7.1. *Dopo la k -esima passata completa, d_j non supera il costo del miglior cammino da s a j che usa al più k archi; nella versione sincrona coincide con tale costo.*

Un cammino semplice contiene al più $n - 1$ archi. Se dopo $n - 1$ passate un arco è ancora rilassabile, esiste un ciclo di costo negativo raggiungibile da s .

1. Inizializzare distanze e predecessori.

2. Per $k = 1, \dots, n - 1$, rilassare tutti gli archi.
3. Se una passata non modifica alcuna etichetta, terminare anticipatamente.
4. Eseguire un'ulteriore scansione: un miglioramento segnala un ciclo negativo.

La complessità è $O(nm)$. Per problemi da tutte le sorgenti, Bellman-Ford può essere combinato con la riponderazione di Johnson; per grafi densi si usa anche Floyd-Warshall, di complessità $O(n^3)$.

8 Cicli negativi e illimitatezza

Occorre distinguere tre situazioni:

- un ciclo negativo non raggiungibile da s non influenza le distanze da s ;
- un ciclo negativo raggiungibile da s rende non definita la distanza finita verso tutti i nodi raggiungibili dal ciclo;
- nel problema specifico $s-t$, l'illimitatezza richiede anche che t sia raggiungibile dal ciclo.

Questa distinzione coincide con l'interpretazione come flusso a costo minimo: una circolazione negativa può essere ripetuta senza cambiare i bilanci, facendo diminuire indefinitamente il costo.

9 Albero dei cammini minimi

Se ogni nodo raggiungibile $j \neq s$ sceglie un predecessore π_j tale che

$$d_j = d_{\pi_j} + c_{\pi_j j},$$

gli archi (π_j, j) formano un'arborescenza uscente radicata in s , detta albero dei cammini minimi, purché non si introducano cicli di predecessori a costo nullo. In caso di distanze multiple l'albero non è unico, anche se il vettore delle distanze lo è.

I vincoli duali si possono riscrivere come costi ridotti

$$\gamma_{ij} = c_{ij} + d_i - d_j \geq 0.$$

Sugli archi dell'albero scelto vale $\gamma_{ij} = 0$: è la stessa condizione incontrata nel semplice so su reti.

10 Esempio comparativo

Consideriamo gli archi

$$(s, a) : 4, quad(s, b) : 2, quad(b, a) : 1, quad(a, t) : 3, quad(b, t) : 7.$$

Dijkstra rende permanente prima b con distanza 2, poi rilassa (b, a) ottenendo $d_a = 3$, quindi trova $d_t = 6$ tramite $s - b - a - t$. I predecessori sono $\pi_b = s$, $\pi_a = b$, $\pi_t = a$.

Se il costo di (b, a) diventa -3 , lo stesso cammino costa $2 - 3 + 3 = 2$, ma l'uso di Dijkstra non è più giustificato. Bellman-Ford gestisce correttamente il costo negativo e conferma la distanza 2 se non vi sono cicli negativi.

11 Python e NetworkX

Python/NetworkX

NetworkX usa di norma l'attributo `weight`. `single_source_dijkstra` richiede pesi non negativi; `single_source_bellman_ford` ammette pesi negativi; `shortest_path` può scegliere il metodo tramite il parametro `method`.

Listing 1: Distanze, cammini e controllo con NetworkX

```
import networkx as nx

G = nx.DiGraph()
G.add_weighted_edges_from([
    ("s", "a", 4), ("s", "b", 2),
    ("b", "a", 1), ("a", "t", 3),
    ("b", "t", 7)
])

dist, paths = nx.single_source_dijkstra(G, "s", weight="weight")
print(dist["t"], paths["t"])

# Versione che consente pesi negativi
dist_bf, paths_bf = nx.single_source_bellman_ford(
    G, "s", weight="weight"
)
assert dist == dist_bf
```

Per un DAG è possibile iterare direttamente sui nodi restituiti da `nx.topological_sort`. Se un nodo non è raggiungibile, NetworkX può ometterlo dai dizionari oppure segnalare l'assenza di un cammino nelle interrogazioni punto-punto.

12 Registro delle notazioni

Registro delle notazioni

Simbolo	Significato
s, t	origine e destinazione
c_{ij}	costo dell'arco (i, j)
$P, C(P)$	cammino e suo costo
d_i, d_i^*	etichetta corrente e distanza minima da s
π_i	predecessore del nodo i
S, T	nodi permanenti e provvisori in Dijkstra
γ_{ij}	$c_{ij} + d_i - d_j$, costo ridotto
$+\infty$	nodo non ancora raggiunto

13 Errori frequenti

1. Cambiare segno a b : per un'unità da s a t vale $b_s = -1$, $b_t = 1$.
2. Confondere distanza e numero di archi del cammino.
3. Applicare Dijkstra in presenza di costi negativi.

4. Aggiornare la distanza senza aggiornare il predecessore.
5. Dichiarare un ciclo negativo rilevante senza verificarne la raggiungibilità.
6. Dimenticare che più cammini minimi possono avere lo stesso costo.
7. Trattare un nodo non raggiungibile come se avesse distanza zero.
8. Confondere un albero dei cammini minimi con un albero ricoprente di costo minimo.

14 Esercizi

Esercizio 7.1 – Formulazione. Scrivere primale e duale del cammino minimo da s a t e verificare la debole dualità lungo un cammino assegnato.

Esercizio 7.2 – DAG. Numerare topologicamente un DAG e calcolare distanze e predecessori da una sorgente, ammettendo anche costi negativi.

Esercizio 7.3 – Dijkstra. Eseguire una tabella delle etichette provvisorie e permanenti e ricostruire l'albero dei cammini minimi.

Esercizio 7.4 – Bellman-Ford. Mostrare le etichette dopo ogni passata e verificare le equazioni di Bellman alla terminazione.

Esercizio 7.5 – Ciclo negativo. Aggiungere un arco all'istanza dell'Esercizio 7.4 in modo da creare un ciclo negativo raggiungibile e individuarlo tramite l'ulteriore scansione.

Esercizio 7.6 – NetworkX. Confrontare Dijkstra e Bellman-Ford su un grafo a pesi non negativi, poi introdurre un peso negativo e verificare quale metodo rimane applicabile.

15 Sintesi del capitolo

Obiettivi di apprendimento

- Il cammino minimo si formula inviando un'unità di flusso da s a t .
- Le variabili duali sono etichette di distanza e soddisfano i vincoli di Bellman.
- Il rilassamento migliora un'etichetta e registra il relativo predecessore.
- Nei DAG basta un passaggio in ordine topologico, anche con costi negativi.
- Dijkstra richiede costi non negativi; Bellman-Ford rileva cicli negativi.
- Gli archi dell'albero dei cammini minimi hanno costo ridotto nullo.

Verso il capitolo successivo. Le relazioni di precedenza tra attività formano naturalmente un DAG. Nel CPM le distanze e i cammini di lunghezza massima diventeranno tempi di completamento e cammini critici.